Trabajo Práctico Integrador — Parte 2

Redmine sobre AWS: EC2 con el rol de Ansible y PostgreSQL en RDS, gestionado con OpenTofu

Santiago Marcos

6 de noviembre de 2026

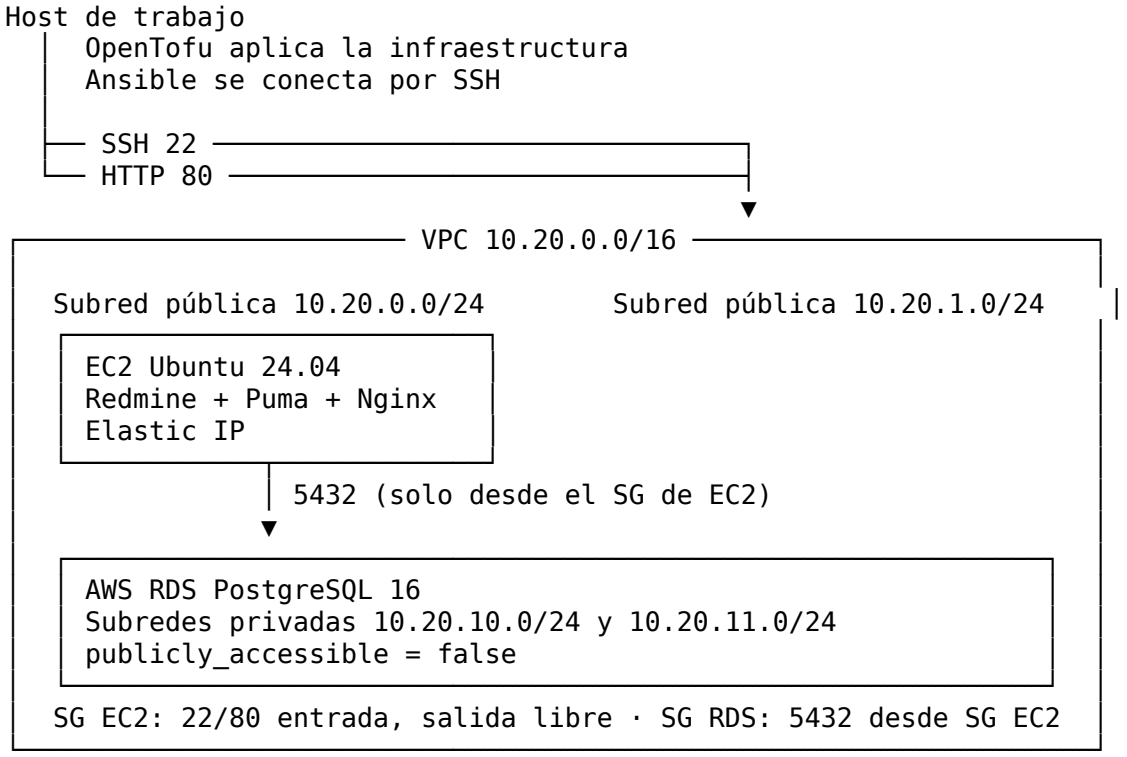
1. Objetivo y alcance

Esta parte reutiliza el rol de Redmine de la parte 1, pero cambia la infraestructura: la aplicación corre en una **instancia EC2** de AWS y la base de datos deja de instalarse con Ansible para usar **AWS RDS PostgreSQL**. Todos los recursos de AWS se declaran y gestionan con **OpenTofu**.

El alcance incluye:

- VPC, subredes públicas y privadas, Internet Gateway y tablas de ruteo.
- Security Groups para EC2 y RDS.
- Key Pair, instancia EC2 Ubuntu 24.04 y Elastic IP.
- Instancia RDS PostgreSQL privada, cifrada y sin acceso público.
- Inventario de Ansible generado desde los outputs de OpenTofu.
- Playbook `playbooks/aws.yml`, que ejecuta los roles de la aplicación **sin** `redmine_database`.
- Verificación, documentación, linters y limpieza de recursos.

2. Arquitectura



La aplicación conserva la misma arquitectura interna que en Vagrant: Nginx recibe el tráfico, sirve los estáticos y deriva el resto a Puma por socket UNIX. Lo único que cambia es el origen de la base: RDS en lugar de PostgreSQL local.

3. Recursos gestionados por OpenTofu

Archivo	Recursos declarados
`network.tf`	VPC, Internet Gateway, subredes públicas/privadas y route tables.
`security.tf`	Security Group de EC2 y Security Group de RDS.
`compute.tf`	Key Pair, instancia EC2, user-data y Elastic IP.
`rds.tf`	DB Subnet Group e instancia RDS PostgreSQL.
`locals.tf`	Tags comunes, zonas de disponibilidad y AMI Ubuntu 24.04.
`variables.tf`	Región, red, tamaños, credenciales y opciones de RDS.
`providers.tf`	Provider de AWS y tags por defecto.
`outputs.tf`	IP pública, endpoint de RDS, URL y comando SSH.

La AMI se obtiene dinámicamente del catálogo oficial de Canonical (`ubuntu-noble-24.04-amd64-server-*`). La instancia EC2 se crea en la primera subred pública y recibe una Elastic IP. El disco raíz es gp3 y está cifrado.

4. Decisiones de seguridad y de diseño

- **RDS privada:** `publicly_accessible = false` y ubicada en subredes privadas.
- **Acceso a la base:** el Security Group de RDS solo acepta el puerto 5432 desde el Security Group de EC2, no desde Internet.
- **Credenciales:** la contraseña de RDS se pasa por `TF_VAR_db_password`; no se escribe en `terraform.tfvars` ni en Git. El inventario de Ansible la lee con `lookup('env', 'TF_VAR_db_password')`.
- **Estado:** `tofu/*.tfstate` y `inventory/aws/hosts.yml` están ignorados por Git.
- **EC2:** exige IMDSv2 (`http_tokens = "required"`), tiene volúmenes cifrados y solo abre SSH y HTTP según los CIDR configurados.
- **RDS:** almacenamiento cifrado, backups y protección contra borrado configurables; en el laboratorio se desactivan el snapshot final y la protección para poder destruir sin fricción.

La contraseña maestra de RDS se valida en OpenTofu con un mínimo de 8 caracteres. Para un entorno real se recomienda un backend remoto para el estado y AWS Secrets Manager o Ansible Vault para el secreto.

5. Flujo de despliegue

5.1 Configuración inicial

```
cp tofu/terraform.tfvars.example tofu/terraform.tfvars
$EDITOR tofu/terraform.tfvars
export TF_VAR_db_password='una-contraseña-larga'
```

Variables principales:

- Región: `aws_region` (`us-east-1`).
- Proyecto y entorno: `project` (`redmine`) y `environment` (`lab`).
- Cómputo: `instance_type` (`t3.micro`).

- Base: `db_instance_class` (`db.t3.micro`) y `db_engine_version` (16).
- Acceso: rutas de claves SSH y `ssh_cidr_blocks`.

Se recomienda restringir `ssh_cidr_blocks` a la IP propia.

5.2 Crear la infraestructura

```
tofu -chdir=tofu init
tofu -chdir=tofu validate
tofu -chdir=tofu plan
tofu -chdir=tofu apply
```

El despliegue se completa con comandos manuales de OpenTofu y Ansible. El único script es `scripts/aws-inventory.sh`, que traduce los outputs al inventario.

5.3 Generar el inventario

El inventario se construye desde los outputs y no contiene la contraseña:

```
./scripts/aws-inventory.sh
```

```yaml
all:
  children:
    aws:
      hosts:
        redmine:
          ansible_host: "<ec2_public_ip>"
          ansible_user: "ubuntu"
          ansible_ssh_private_key_file: "<ssh_private_key_path>"
          redmine_host_name: "<ec2_public_ip>"
          redmine_db_host: "<rds_address>"
          redmine_db_port: 5432
          redmine_db_name: "redmine"
          redmine_db_user: "redmine"
          redmine_db_sslmode: "require"
          redmine_verify_local_postgresql: false
          redmine_db_password: "{{ lookup('env', 'TF_VAR_db_password') }}"
```

5.4 Ejecutar Ansible

`playbooks/aws.yml` primero espera a que la instancia acepte SSH y a que `cloud-init` termine. Después ejecuta, con `become`:

1. `redmine_common`
2. `redmine_ruby`
3. `redmine`
4. `redmine_nginx`

No ejecuta `redmine_database`, porque PostgreSQL ya existe en RDS.

```
export TF_VAR_db_password='una-contraseña-larga'
ansible-playbook -i inventory/aws/hosts.yml playbooks/aws.yml
```

5.5 Verificar y acceder

```
export TF_VAR_db_password='una-contraseña-larga'
ansible-playbook -i inventory/aws/hosts.yml playbooks/verify.yml
```

La verificación es la misma que en Vagrant, con dos diferencias: la conexión a PostgreSQL apunta a RDS con `sslmode=require` y se omite la comprobación del servicio PostgreSQL local con `redmine_verify_local_postgresql: false`.

La URL y el acceso SSH se obtienen con:

```
tofu -chdir=tofu output -raw redmine_url
tofu -chdir=tofu output -raw ssh_command
```

5.6 Destruir

```
export TF_VAR_db_password='una-contraseña-larga'
tofu -chdir=tofu destroy
rm -f inventory/aws/hosts.yml
```

Se destruyen EC2, RDS, red, Elastic IP y demás recursos, y se elimina el inventario generado.

6. Adaptaciones de Ansible para RDS

El rol de Redmine se reutiliza casi sin cambios. Las adaptaciones son mínimas y están parametrizadas:

- `redmine_db_host` y `redmine_db_port` apuntan al endpoint y puerto de RDS.
- `redmine_db_sslmode: require` agrega `sslmode` a `config/database.yml` y al módulo de consulta de PostgreSQL.
- `redmine_verify_local_postgresql: false` evita exigir PostgreSQL local en la EC2 durante `verify.yml`.
- `playbooks/aws.yml` omite el rol `redmine_database`.
- La contraseña se inyecta en tiempo de ejecución desde el entorno.

El resto de la configuración —Ruby 4.0.9 con rbenv, usuario `redmine`, Puma por socket UNIX, Nginx como proxy inverso, estáticos servidos por Nginx y servicios habilitados— es exactamente la misma que en la parte 1.

7. Verificación y evidencias

La infraestructura no se dejó desplegada para no generar costos: el estado de OpenTofu quedó vacío y no hay recursos activos. El flujo se validó de forma estática y con el mismo código de Ansible verificado en Vagrant.

Comprobación	Resultado
`tofu fmt -check -recursive tofu`	Sin diferencias
`tofu init -backend=false`	Inicialización correcta
`tofu validate`	`Success! The configuration is valid.`
`yamllint .`	Sin errores
`ansible-lint` (production)	0 fallos, 0 warnings
`bash -n scripts/*.sh`	Sin errores de sintaxis
`molecule test`	Escenario completo con idempotencia y verify
`tofu state list`	Sin recursos (entorno limpio)

La validación de OpenTofu cubre provider, variables, VPC, subredes, Security Groups, EC2, Elastic IP, RDS y outputs. La validación de Ansible cubre el playbook `aws.yml`, el inventario generado y los roles compartidos con Vagrant.

8. Comparación con la parte 1

Tema	Vagrant (parte 1)	AWS (parte 2)
Infraestructura	VirtualBox + Vagrant	OpenTofu
Base de datos	Rol `redmine_database` local	RDS PostgreSQL 16
Playbook	`playbooks/site.yml`	`playbooks/aws.yml`
Conexión a base	`127.0.0.1:5432`	Endpoint de RDS, `sslmode=require`
Acceso	`127.0.0.1:8080`	Elastic IP en puerto 80
Sistema operativo	Box `mw-ubuntu-24.04`	AMI oficial Ubuntu 24.04
Aplicación	Roles `redmine_*`	Los mismos roles, sin `redmine_database`

9. Documentación revisada

Se revisó la documentación existente contra el código y se corrigieron los puntos que no coincidían con el estado real:

- `README.md`: se agregó el uso de puertos alternativos y una tabla de estado verificado en el laboratorio.
- `flujo.md`: se actualizó la descripción de Molecule y el estado final comprobado.
- `docs/aws.md`: se documentó la espera de SSH antes de `cloud-init`, el flujo de inventario, las variables y la destrucción de recursos.
- `.github/workflows/ci.yml`: el job de Molecule ahora instala las colecciones antes de ejecutar el escenario.
- `molecule/default/requirements.yml`: declara las colecciones para que la prueba sea autocontenida.
- `.gitignore`: se amplió para excluir estados, planes y artefactos generados.

10. Conclusión

La variante de AWS cumple el objetivo: una instancia EC2 ejecuta el mismo rol de Redmine de la parte 1 y usa RDS PostgreSQL en lugar de una base local. Toda la infraestructura se gestiona con OpenTofu, el acceso a la base queda restringido por Security Groups, las credenciales no se versionan y el flujo de despliegue, verificación y destrucción queda documentado como comandos reproducibles. La entrega está lista para demostrarse con la URL de la Elastic IP.

El código es verificable y probable, se encuentra disponible en el siguiente repositorio (se utilizó el mismo para Parte 1 y parte 2, ver commit correspondiente): https://github.com/santi440/ansible-redmine